Spark - ML

m2iformation.fr

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

ML avec Spark MLlib

Utopios® Tous droits réservés

Sommaire

- Pipeline MLlib & Préparation des données distribuées
- Régression et Classification distribuée
- Comparaison Spark MLlib vs Scikit-learn

Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Pipeline MLlib & Préparation des données distribuées

Utopios® Tous droits réservés

Qu'est-ce qu'Apache Spark ?

Apache Spark est un moteur de calcul distribué open-source conçu pour traiter de très grandes volumétries de données.
Contrairement à Hadoop MapReduce, Spark effectue les calculs **en mémoire (in-memory)**, ce qui le rend **10 à 100 fois plus rapide** pour de nombreux scénarios.

Utopios® Tous droits réservés

Composants clés de l'écosystème Spark

- **Spark Core** : moteur d'exécution de base, gestion des RDD
- **Spark SQL / DataFrames** : requêtes SQL sur données distribuées
- **Spark Streaming** : traitement de flux en temps réel
- **MLlib** : bibliothèque de Machine Learning distribuée
- **GraphX** : traitement de graphes

Utopios® Tous droits réservés

Pourquoi MLlib ?

Spark MLlib est la bibliothèque ML intégrée à Spark. Elle propose deux API :

- `spark.mllib` **(RDD-based)** : l'ancienne API, stable mais moins recommandée
- `spark.ml` **(DataFrame-based)** : l'API moderne recommandée, compatible avec les Pipelines

Utopios® Tous droits réservés

Avantages clés :

- **Scalabilité horizontale** : traite des milliards de lignes en ajoutant des nœuds
- **Pipeline intégré** : enchaîne les étapes de pré-traitement et d'entraînement
- **Reproductibilité** : sauvegarde et rechargement des modèles
- **Compatibilité** : lit CSV, Parquet, JSON, ORC, HDFS, S3, Azure Blob…

Utopios® Tous droits réservés

Architecture Docker pour Spark

Nous allons utiliser Docker Compose pour simuler un cluster Spark en local avec un **master** et des **workers**.

```yaml
version: "3.8"
services:
  spark-master:
    image: bitnami/spark:3.5
    environment:
      - SPARK_MODE=master
    ports:
      - "8080:8080" # Spark UI
      - "7077:7077" # Spark Master port
    volumes:
      - ./notebooks:/opt/bitnami/spark/notebooks
      - ./data:/opt/bitnami/spark/data
```

Utopios® Tous droits réservés

Architecture Docker pour Spark (suite)

```yaml
spark-worker-1:
  image: bitnami/spark:3.5
  environment:
    - SPARK_MODE=worker
    - SPARK_MASTER_URL=spark://spark-master:7077
    - SPARK_WORKER_MEMORY=2G
    - SPARK_WORKER_CORES=2
  depends_on:
    - spark-master
jupyter:
  image: jupyter/pyspark-notebook:latest
  ports:
    - "8888:8888"
  environment:
    - SPARK_MASTER=spark://spark-master:7077
  volumes:
```

Utopios® Tous droits réservés

Commandes de démarrage

```
# Lancer le cluster
docker-compose up -d

# Vérifier l'état
docker-compose ps

# Spark UI accessible sur : http://localhost:8080
# Jupyter Lab  accessible sur : http://localhost:8888

# Arrêter le cluster
docker-compose down
```

Utopios® Tous droits réservés

Création d'une SparkSession

La `SparkSession` est le **point d'entrée unique** pour toutes les fonctionnalités Spark. Elle remplace `SparkContext`, `SQLContext` et `HiveContext`.

```
from pyspark.sql import SparkSession

# Mode cluster Docker
spark = SparkSession.builder \
    .appName('MLlib_Formation') \
    .master('spark://spark-master:7077') \
    .config('spark.executor.memory', '2g') \
    .config('spark.executor.cores', '2') \
    .config('spark.sql.shuffle.partitions', '8') \
    .getOrCreate()
```

Utopios® Tous droits réservés

DataFrames Spark vs Pandas

Caractéristique	Pandas DataFrame	Spark DataFrame
Stockage	Mémoire locale (1 machine)	Distribué (N machines)
Volume	< RAM disponible	Pétaoctets
Exécution	Eagère (immédiate)	Paresseuse (lazy)
Indexation	Index numérique	Pas d'index
Modification	In-place possible	Immuable (nouveau DF)

Utopios® Tous droits réservés

Chargement et exploration des données

```
# Lecture d'un CSV
df = spark.read \
    .option('header', 'true') \
    .option('inferSchema', 'true') \
    .option('sep', ',') \
    .csv('/data/titanic.csv')

# Informations de base
df.printSchema()
df.show(5)
df.count()

# Statistiques descriptives
df.describe().show()
```

Utopios® Tous droits réservés

Lecture Parquet (recommandé en production)

```
# Parquet : format columaire optimisé pour Spark
# Avantages : compression, pruning de colonnes, predicate pushdown

# Écriture en Parquet
df.write.mode('overwrite').parquet('/data/titanic.parquet')

# Lecture depuis Parquet
df_parquet = spark.read.parquet('/data/titanic.parquet')

# Lecture avec partitionnement (très efficace pour gros volumes)
df.write \
    .mode('overwrite') \
    .partitionBy('Pclass') \
    .parquet('/data/titanic_partitioned.parquet')
```

Utopios® Tous droits réservés

Vue d'ensemble des Transformers MLlib

Transformer	Utilité	Exemple d'usage
`StringIndexer`	Convertit catégories → index numériques	Sexe: male=0, female=1
`OneHotEncoder`	Index → vecteur one-hot	Embarked: [1,0,0], [0,1,0]...
`VectorAssembler`	Assemble colonnes en vecteur feature	Toutes features → `features`
`StandardScaler`	Standardise (moyenne=0, std=1)	Age, Fare sur même échelle
`MinMaxScaler`	Normalise dans [0,1]	Normalisation pour distance
`Imputer`	Remplace valeurs manquantes	Age → moyenne/médiane
`Bucketizer`	Discrétise valeurs continues	Age → tranches [0-18, 18-60, 60+]

Utopios® Tous droits réservés

StringIndexer

```
# StringIndexer : convertit une colonne string en index numériques
# L'index 0 est attribué à la valeur la plus fréquente
sex_indexer = StringIndexer(
    inputCol='Sex',
    outputCol='Sex_indexed',
    handleInvalid='keep'  # 'keep' garde les inconnus, 'skip' les supprime
)

sex_model = sex_indexer.fit(df)
df_indexed = sex_model.transform(df)

# Indexer plusieurs colonnes en une seule fois
multi_indexer = StringIndexer(
    inputCols=['Sex', 'Embarked'],
    outputCols=['Sex_idx', 'Embarked_idx'],
```

Utopios® Tous droits réservés

OneHotEncoder

```
# OneHotEncoder convertit des index numériques en vecteurs binaires
# ATTENTION : nécessite un StringIndexer en amont !
ohe = OneHotEncoder(
    inputCols=['Sex_idx', 'Embarked_idx'],
    outputCols=['Sex_ohe', 'Embarked_ohe'],
    dropLast=True  # Évite la multicolinéarité (k-1 colonnes pour k catégories)
)

ohe_model = ohe.fit(df_multi)
df_ohe = ohe_model.transform(df_multi)

# Exemple de résultat :
# Sex: male   -> (1, [0], [1.0])   (vecteur sparse de dim 1)
# Sex: female -> (1, [], [])        (vecteur nul = 1 catégorie supprimée)
df_ohe.select('Sex', 'Sex_idx', 'Sex_ohe').show(5)
```

Utopios® Tous droits réservés

Imputer

```
# Imputer remplace les valeurs null/NaN par une statistique
imputer = Imputer(
    inputCols=['Age', 'Fare'],
    outputCols=['Age_imputed', 'Fare_imputed'],
    strategy='median'  # 'mean', 'median', ou 'mode'
)

imputer_model = imputer.fit(df)
df_imputed = imputer_model.transform(df)
```

Utopios® Tous droits réservés

VectorAssembler

```
# VectorAssembler combine plusieurs colonnes en un seul vecteur dense
# C'est la dernière étape avant l'entraînement d'un modèle
assembler = VectorAssembler(
    inputCols=[
        'Pclass', 'Age_imputed', 'SibSp', 'Parch', 'Fare_imputed',
        'Sex_ohe', 'Embarked_ohe'
    ],
    outputCol='features',
    handleInvalid='skip'
)

df_assembled = assembler.transform(df_ohe)

# Résultat : une colonne 'features' de type Vector
df_assembled.select('features').show(3, truncate=False)
```

Utopios® Tous droits réservés

StandardScaler & MinMaxScaler

```
# StandardScaler : standardise le vecteur features
# (soustrait la moyenne, divise par l'écart-type)
scaler = StandardScaler(
    inputCol='features', outputCol='features_scaled',
    withMean=True,    # centre sur la moyenne
    withStd=True      # réduit par l'écart-type
)
scaler_model = scaler.fit(df_assembled)
df_scaled = scaler_model.transform(df_assembled)

# MinMaxScaler : normalise dans [0, 1]
mm_scaler = MinMaxScaler(
    inputCol='features', outputCol='features_normalized',
    min=0.0, max=1.0)
mm_model = mm_scaler.fit(df_assembled)
```

Utopios® Tous droits réservés

Concept du Pipeline MLlib

Un **Pipeline MLlib** est une séquence ordonnée d'étapes (Stages) qui chaîne automatiquement les transformations et l'entraînement.

Pourquoi utiliser un Pipeline ?

- Évite de ré-appliquer manuellement chaque étape de transformation
- Garantit que les paramètres appris sont **identiques** entre train et test
- Simplifie la sauvegarde et le rechargement du modèle complet
- Facilite la mise en production (**1 seul objet** à déployer)

Utopios® Tous droits réservés

Construction d'un Pipeline complet

```
sex_indexer = StringIndexer(inputCol='Sex', outputCol='Sex_idx', handleInvalid='keep')
embarked_indexer = StringIndexer(inputCol='Embarked', outputCol='Embarked_idx', handleInvalid='keep')
ohe = OneHotEncoder(inputCols=['Sex_idx', 'Embarked_idx'],
                    outputCols=['Sex_ohe', 'Embarked_ohe'], dropLast=True)
imputer = Imputer(inputCols=['Age', 'Fare'], outputCols=['Age_imp', 'Fare_imp'], strategy='median')
assembler = VectorAssembler(
    inputCols=['Pclass', 'Age_imp', 'SibSp', 'Parch', 'Fare_imp', 'Sex_ohe', 'Embarked_ohe'],
    outputCol='features', handleInvalid='skip')
scaler = StandardScaler(inputCol='features', outputCol='features_scaled',
                        withMean=True, withStd=True)

pipeline = Pipeline(stages=[sex_indexer, embarked_indexer, ohe, imputer, assembler, scaler])
```

Utopios® Tous droits réservés

Fit, Transform et Sauvegarde du Pipeline

```
# Split train/test
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)

pipeline_model = pipeline.fit(train_df)

# Transform sur train et test
train_prepared = pipeline_model.transform(train_df)
test_prepared  = pipeline_model.transform(test_df)

# Sauvegarde du pipeline
pipeline_model.save('/models/preprocessing_pipeline')

pipeline_loaded = PipelineModel.load('/models/preprocessing_pipeline')
```

Utopios® Tous droits réservés

Régression et Classification distribuée

Utopios® Tous droits réservés

Vue d'ensemble des algorithmes de Régression

Algorithme	Cas d'usage	Paramètre clé
`LinearRegression`	Relation linéaire, baseline	`regParam`, `elasticNetParam`
`DecisionTreeRegressor`	Interprétable, non-linéaire	`maxDepth`, `minInstancesPerNode`
`RandomForestRegressor`	Robuste, généraliste	`numTrees`, `maxDepth`, `featureSubsetStrategy`
`GBTRegressor`	Haute précision (boosting)	`maxIter`, `stepSize`, `maxDepth`
`GeneralizedLinearRegression`	Poisson, Gamma, Tweedie	`family`, `link`, `regParam`

Utopios® Tous droits réservés

Régression Linéaire

```
lr = LinearRegression(
    featuresCol='features_scaled',
    labelCol='SalePrice',
    maxIter=100,
    regParam=0.1,
    elasticNetParam=0.0  # 0 = Ridge, 1 = Lasso
)

pipeline_lr = Pipeline(stages=[...preprocessing..., lr])
model_lr = pipeline_lr.fit(train_df)

evaluator = RegressionEvaluator(labelCol='SalePrice', predictionCol='prediction')
print("RMSE:", evaluator.evaluate(predictions, {evaluator.metricName: 'rmse'}))
print("R²  :", evaluator.evaluate(predictions, {evaluator.metricName: 'r2'}))
print("MAE :", evaluator.evaluate(predictions, {evaluator.metricName: 'mae'}))
```

Utopios® Tous droits réservés

Random Forest Regressor

```
rf = RandomForestRegressor(
    featuresCol='features_scaled',
    labelCol='SalePrice',
    numTrees=100,
    maxDepth=10,
    featureSubsetStrategy='auto',
    seed=42
)

pipeline_rf = Pipeline(stages=[...preprocessing..., rf])
model_rf = pipeline_rf.fit(train_df)

# Importance des features
rf_model = model_rf.stages[-1]
print(rf_model.featureImportances)
```

Utopios® Tous droits réservés

Gradient Boosting Trees (GBT)

```
gbt = GBTRegressor(
    featuresCol='features_scaled',
    labelCol='SalePrice',
    maxIter=50,
    stepSize=0.1,
    maxDepth=5,
    subsamplingRate=0.8,
    seed=42
)

pipeline_gbt = Pipeline(stages=[...preprocessing..., gbt])
model_gbt = pipeline_gbt.fit(train_df)
```

Utopios® Tous droits réservés

Vue d'ensemble des algorithmes de Classification

Algorithme	Particularité	Sortie
`LogisticRegression`	Binaire & multiclasse, probabiliste	`rawPrediction`, `probability`, `prediction`
`DecisionTreeClassifier`	Interprétable, règles visibles	`prediction`, `probability`
`RandomForestClassifier`	Robuste, gère l'overfitting	`prediction`, `probability`
`GBTClassifier`	Haute précision (binaire seulement)	`rawPrediction`, `prediction`
`NaiveBayes`	Texte, données sparses	`prediction`, `probability`
`LinearSVC`	Grands jeux de données binaires	`rawPrediction`, `prediction`

Utopios® Tous droits réservés

Régression Logistique

```
from pyspark.ml.classification import LogisticRegression

lr_clf = LogisticRegression(
    featuresCol='features_scaled',
    labelCol='Survived',
    maxIter=100,
    regParam=0.1,
    elasticNetParam=0.0,
    family='binomial'
)

pipeline_clf = Pipeline(stages=[...preprocessing..., lr_clf])
model_clf = pipeline_clf.fit(train_df)

predictions = model_clf.transform(test_df)
```

Utopios® Tous droits réservés

Métriques d'évaluation pour la Classification

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator, MulticlassClassificationEvaluator

# AUC-ROC
binary_eval = BinaryClassificationEvaluator(labelCol='Survived', metricName='areaUnderROC')
print("AUC-ROC:", binary_eval.evaluate(predictions))

# Accuracy, F1
multi_eval = MulticlassClassificationEvaluator(labelCol='Survived', predictionCol='prediction')
print("Accuracy:", multi_eval.evaluate(predictions, {multi_eval.metricName: 'accuracy'}))
print("F1-Score:", multi_eval.evaluate(predictions, {multi_eval.metricName: 'f1'}))

# Matrice de confusion
predictions.groupBy('Survived', 'prediction').count().show()
```

Utopios® Tous droits réservés

Random Forest Classifier

```
from pyspark.ml.classification import RandomForestClassifier

rf_clf = RandomForestClassifier(
    featuresCol='features_scaled',
    labelCol='Survived',
    numTrees=100,
    maxDepth=8,
    featureSubsetStrategy='sqrt',
    seed=42
)

pipeline_rf_clf = Pipeline(stages=[...preprocessing..., rf_clf])
model_rf_clf = pipeline_rf_clf.fit(train_df)

# Importance des features
rf_clf_model = model_rf_clf.stages[-1]
```

Utopios® Tous droits réservés

GBT Classifier

```
from pyspark.ml.classification import GBTClassifier

gbt_clf = GBTClassifier(
    featuresCol='features_scaled',
    labelCol='Survived',
    maxIter=50,
    stepSize=0.1,
    maxDepth=5,
    seed=42
    # Note : GBTClassifier ne supporte que la classification BINAIRE
)

pipeline_gbt_clf = Pipeline(stages=[...preprocessing..., gbt_clf])
model_gbt_clf = pipeline_gbt_clf.fit(train_df)
```

Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Comparaison Spark MLlib vs Scikit-learn

Utopios® Tous droits réservés

Comparaison conceptuelle

Dimension	Scikit-learn	Spark MLlib
Architecture	Single machine, in-memory	Distribué, multi-nœuds
Volume max	< RAM du serveur (~100 Go)	Illimité (scale horizontalement)
API	`fit()` / `predict()`	`fit()` / `transform()`
Données entrée	NumPy array, Pandas DF	Spark DataFrame
Algorithmes	>100 algorithmes	~30 algorithmes clés
Pipeline	`sklearn.pipeline.Pipeline`	`pyspark.ml.Pipeline`
Cross-validation	`GridSearchCV`, `RandomizedSearchCV`	`CrossValidator`, `TrainValidationSplit`
Sérialisation	`pickle` / `joblib`	`model.save()` / `load()`

Utopios® Tous droits réservés

Quand utiliser quoi ?

Utiliser Scikit-learn si...	Utiliser Spark MLlib si...
Données < 10 Go en RAM	Données > RAM d'une machine
Prototypage rapide, exploration	Pipeline de production à grande échelle
Algorithme très spécifique non dispo MLlib	Traitement distribué déjà en place
Équipe sans expertise Spark	Besoin de scalabilité horizontale
Machine individuelle, ressources limitées	Cluster disponible (cloud, on-prem)
Modèles complexes (NLP avancé, DL)	Données en streaming temps réel

Utopios® Tous droits réservés

Tableau de synthèse du benchmark

Métrique	Scikit-learn	Spark MLlib
Temps d'entraînement (petits données)	Très rapide (~0.5s)	Plus lent (overhead Spark)
Temps d'entraînement (gros volumes)	Bloque / Out of Memory	Linéaire avec les données
Précision (même algo)	Identique ou très proche	Identique ou très proche
Utilisation CPU/RAM	1 machine, 1 processus	Distribué N machines
Complexité du code	Faible	Modérée
Point d'équilibre	< 1-5 Go en RAM	> 5-10 Go ou cluster

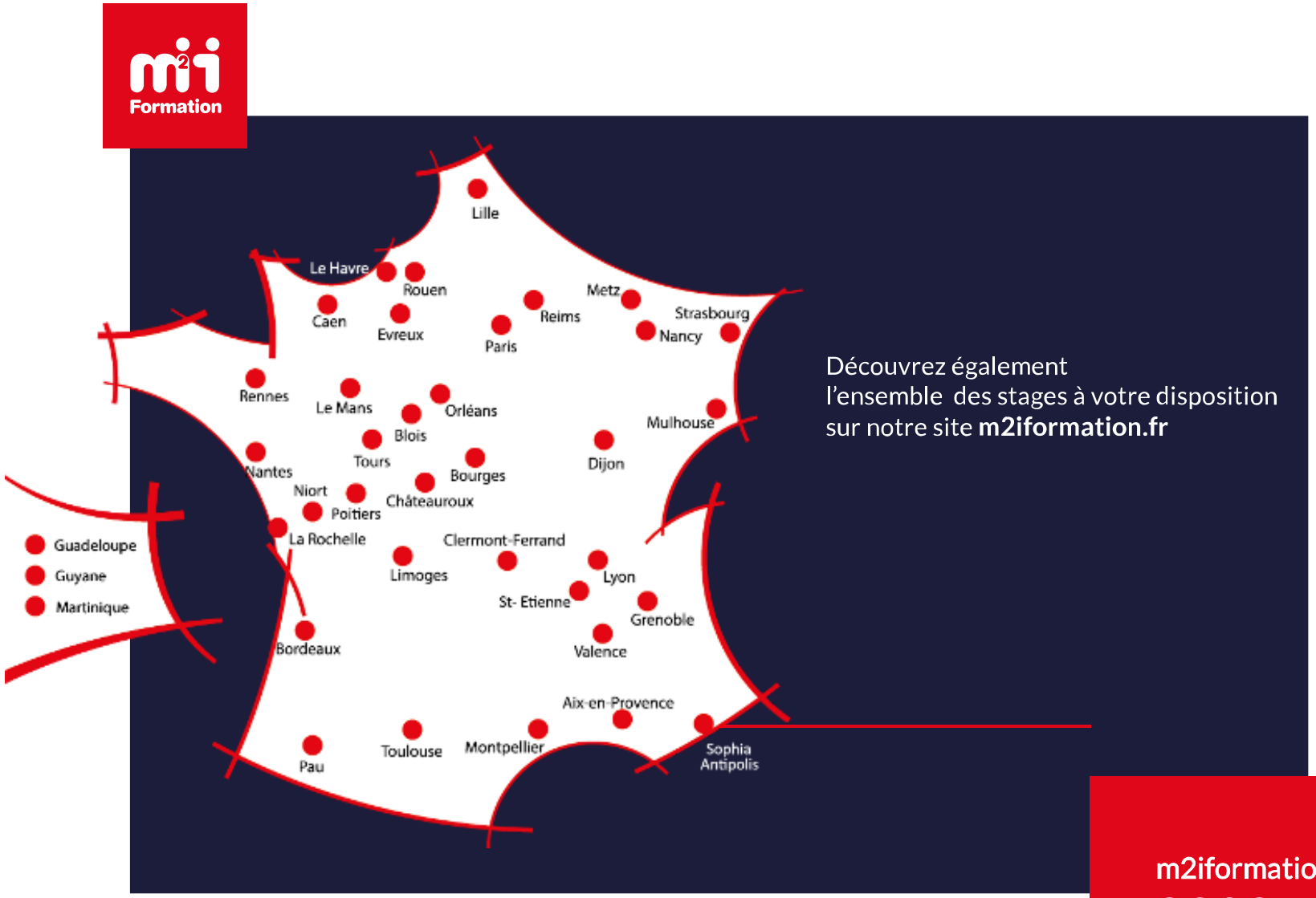
Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Merci pour votre attention

Des questions ?

Utopios® Tous droits réservés



Découvrez également
l'ensemble des stages à votre disposition
sur notre site **m2information.fr**

m2iformation.fr